

ECS Game Engine for Raspberry Pi Pico

Components

AI

AIComponent

Description: Think, Decide, Act architecture with a memory map

How to use: Create Think, Decide, Act functions and map them to the function pointers.
Use the memory map to save data between these functions.

```
AddComponent<AIComponent>();

auto* ai = GetComponent<AIComponent>();
ai->think = [this](Entity* self, float deltaTime)
{
    this->Think(self, deltaTime);
};

ai->decide = [this](Entity* self, float deltaTime)
{
    this->Decide(self, deltaTime);
};

ai->act = [this](Entity* self, float deltaTime)
{
    this->Act(self, deltaTime);
};

if (!myAI) return;

float targetY = 80; //default middle of screen

if (myAI->memory[PongAI::BALL_TO_RIGHT] > 0)
{
    targetY = myAI->memory[PongAI::ESTEMATED_Y];
}

if (targetY < myTransform->GetCenterPos().y - 10)
{
    myAI->memory[PongAI::TARGET_DIR_Y] = -1; //move down
}
else if (targetY > myTransform->GetCenterPos().y + 10)
{
    myAI->memory[PongAI::TARGET_DIR_Y] = 1;
}
else
{
    myAI->memory[PongAI::TARGET_DIR_Y] = 0;
}
```

Core

MovementComponent

Description: Set the speed of the Entity

How to use: Set in Constructor

TimerComponent

Description: Used to keep track of a timer

How to use: Starts at 0

TransformComponent

Description: Keep track of the current Position and Size of the Entity. This size is used for culling in the RenderSystem.

How to use: Set in Constructor

Events

OnEventComponent

Description: Used to save functions for called Events

How to use: Map a function to a EventType

```
auto* onEvent = GetComponent<OnEventComponent>();
onEvent->events[EventType::BALL_OOB] = [this](const Event &event)
{
    this->BallOutOfBounds(event);
};
```

Input

InputIntendComponent

Description: Used to save an Input from Player Input or AI System

How to use: Map InputActions. Also used to allow X and Y movement for Collision Resolving

```
AddComponent<InputIntendComponent>();
auto* intend = GetComponent<InputIntendComponent>();
intend->hasY = true;
```

InputMappingComponent

Description: Used to map Keycode Inputs to InputActions

How to use: Map KeyCodes to directions or actions

```
AddComponent<InputMappingComponent>();

auto* mapping = GetComponent<InputMappingComponent>();
mapping->directionMapping[KEYCODE::UP] = {InputAction::VERTICAL, -speed};
mapping->directionMapping[KEYCODE::DOWN] = {InputAction::VERTICAL, speed};
```

Physics

ColliderComponent

Description: Used to add a axis aligned rectangle collider

How to use: Set in Constructor. Set Offset and Size as well as if it should only Trigger

CollisionResponseComponent

Description: Used to add functions triggered by collisions

How to use: Map functions

```
AddComponent<CollisionResponseComponent>();  
GetComponent<CollisionResponseComponent>()->OnEnter = [this](Entity* self, Entity* other)  
{  
    this->OnCollisionEnter(self, other);  
};
```

PhysicsComponent

Description: Used to keep track of Gravity and current Velocity.

How to use: Set in Constructor. Also select PhysicsType for CollisionResolving

PhysicsMaterialComponent

Description: Used to add Friction and Bounciness factors

How to use: Set in Constructor

Render

AnimationComponent

Description: Used to add Animations

How to use: Map Animations and Transition Functions

```
AddComponent<AnimationComponent>(16, 16);
auto* animation = GetComponent<AnimationComponent>();
animation->AddAnimation(0, AnimationMode::LOOP, 2, 1, "idle.bin");
animation->AddAnimation(1, AnimationMode::LOOP, 4, 4, "walking.bin");

animation->transitions[0].push_back([this](bool& swapped)
{
    auto* physics = GetComponent<PhysicsComponent>();

    if (physics->currentVelocity.Magnitude() > 0.2f)
    {
        GetComponent<AnimationComponent>()->SwapAnimation(1);
        swapped = true;
    }
});

animation->transitions[1].push_back([this](bool& swapped)
{
    auto* physics = GetComponent<PhysicsComponent>();

    if (physics->currentVelocity.Magnitude() < 0.2f)
    {
        GetComponent<AnimationComponent>()->SwapAnimation(0);
        swapped = true;
    }
});
```

CameraComponent

Description: Used to keep track of Camera zoom

How to use: Set in Constructor

FollowTargetComponent

Description: Used (by Camera) to follow another TransformComponent

How to use: Set in Constructor

RectangleComponent

Description: Used to draw a colored Rectangle at TransformComponent

How to use: Set in Constructor

RenderableComponent

Description: Used to set Layer and Parallax Factor. Also, bool if it should be rendered

How to use: Set in Constructor

SpriteComponent

Description: Used to add Sprite

How to use: Set in Constructor

Tiles

TilemapComponent

Description: Add a Tilemap World

How to use: Set in Constructor. Also need to create a Tileset first. Tilemap saved in Textfile with indices.

```
Tileset tileset(32,32,4,"TilesetTest.bin");
tileset.solidTiles[3] = true;

Entity* tiles = new Entity();
tiles->AddComponent<TransformComponent>(Vector2(0,0), Vector2(320, 320)); //size of whole map
tiles->AddComponent<RenderableComponent>(0);
tiles->AddComponent<TilemapComponent>(10, 10, "Tilemap.txt", tileset);
es.DispatchEvent(EventSpawnEntity(tiles));
```

UI

UIButtonComponent

Description: Used for Buttons

How to use: Set in Constructor and then map a Function

```
// --- UI elements ---
auto pongButton = std::make_unique<UIButtonObject>(Vector2(10,30), Vector2(110,20), Color::BLACK, Color::YELLOW, 0);
pongButton->GetComponent<UIButtonComponent>()->OnClicked = [](GameManager& manager)
{
    manager.SwitchGame(std::make_unique<Pong>(manager));
};
myUIElements.emplace_back(std::move(pongButton));
```

UITextComponent

Description: Used to display Text

How to use: Set in Constructor

Systems

AISystem

Description: Uses Think, Decide, Act to handle AI

How to use: In Update with Entity list and deltaTime

Required Components: AIComponent

TimerSystem

Description: Used to measure real Time

How to use: In Update with Entity list and deltaTime

Required Components: TimerComponent

//Could be extended for timed events on countdowns

EventComponentSystem

Description: Used if an Entity has to react to an Event

How to use: Listener to the EventSystem

Required Components: OnEventComponent

InputSystem

Description: Handles hardware input

How to use: In Update with Entity list and deltaTime

Required Components: InputMappingComponent, InputIntendComponent

CollisionSystem

Description: Checks collision between all Entities with a Collider. Handles OnEnter, OnStay, OnExit

How to use: Called in Update with Entity list

Required Components: TransformComponent, ColliderComponent

MovementSystem

Description: Transform Input into velocity adjustment or Lerps Position to a Target Position

How to use: In Update with Entity list and deltaTime

Required Components: TransformComponent, MovementComponent and either PhysicsComponent, InputIntentComponent or FollowTargetComponent

PhysicsSystem

Description: Updates Position with current Velocity and resolves Collisions

How to use: In Update with Entity list and deltaTime, or ResolveCollision with Entity list and deltaTime

Required Components: TransformComponent, PhysicsComponent, MovementComponent; for ResolveCollision ColliderComponent and PhysicsComponent

AnimationSystem

Description: Updates current Animation

How to use: In Update with Entity list and deltaTime

Required Components: AnimationComponent

CameraSystem

Description: Adjust zoom of the Camera

How to use: In Update with Entity list and deltaTime

Required Components: CameraComponent

RenderSystem

Description: Renders all Entities in relation to the current camera,

How to use: In Render with Entity list, Renderer and Camera

Required Components: RenderableComponent, TransformComponent and either, RectangleComponent, SpriteComponent, AnimationComponent or TilemapComponent

UIRenderSystem

Description: Renders UI Elements

How to use: In Render with UI Entity list and Renderer AFTER the (Entity) RenderSystem

Required Components: TransformComponent, either UITextComponent or RectangleComponent

UIUpdateSystem

Description: Updates UI Element like Buttons and on-Screen Timers

How to use: In Update with UI Entity list and deltaTime

Required Components: For Timers TimerComponent and UITextComponent; For buttons UIButtonComponent and RectangleComponent

Manager

EventManager (global Singleton)

Description: Used to send custom Events around the project

How to use: Create the new Event. Send it with the global EventsManager, to receive Events make the class inherit from EventListener and in the Instantiation add it to the EventManager' listener

Required Components: if EventListener then inherit from the EventListener class

DebugManager (global Singleton)

Description: used to display Debug Messages on screen

How to use: It runs with separate rendering after the game rendering, bool PrintLogs needs to be true, it can have 8 messages at a time and will delete the oldest if a 9th gets added

Required Components: none

EntityManager

Description: keeps track of all current Entities and sort for render layers

How to use: when creating a new Entity use the Event "EventSpawnEntities" to add it to the Entity Manager

Required Components: none

GameManager

Description: manages the current selected Game and to switch games

How to use: A game needs to inherit from the Game Class

Required Components: none

ResourceManager (global Singleton)

Description: used to handle file loading, writing and reading from the SD card

How to use: is used to have access to the sdManager, GetSprite Function is used often so it's a function, others will be added

Required Components: none